

DCA0305

Lesson 10 · October 26

morsinaldo.medeiros.075@ufrn.edu.br
ivanovitch.silva@ufrn.br

AI Agents I

From models that talk to systems that act

Where we are

UNIT 1 · THE LLM BLOCK

05 History | 06 Transformer | 07 Ollama | 08 Classic RAG | 09 Advanced RAG

UNIT 2 · AGENTS & THE CLAUDE ECOSYSTEM

10
Agents I
Introduction and Architectures

11
Agents II
MCPs

12
Agents III
Multi-agent

Why agents

A model alone · workflow vs. agent · the agents around you

I

A model, alone, cannot act



Frozen in time

Nothing after the training cutoff



No hands

Describes actions; performs none



One shot

No try, no check, no retry



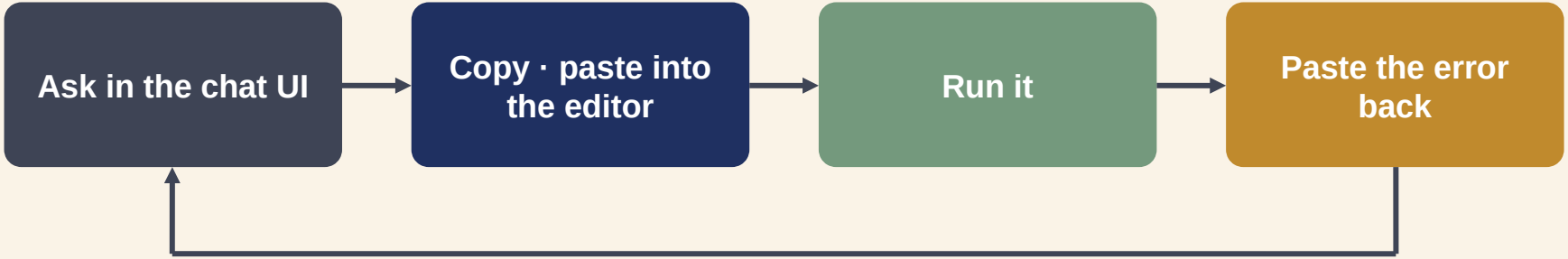
No memory

Every chat starts from zero

RAG gave the model things to read. Today we give it things to do.

2023: you were the loop

How we all coded with ChatGPT and Claude, before agents



…and again, and again

An agent is the same loop, without you as the clipboard.

Workflow or agent?

NOT AGENTIC



The path is hardcoded — your code decides

your RAG pipeline · week 06

AGENTIC



The model decides the next step — and how many

today

An agent uses the LLM as its reasoning engine to choose the sequence of actions.

You already live with agents



Claude Code

an agent in your terminal — L14



Deep Research

minutes-long search loops



Copilot · Cursor

agentic coding in the IDE

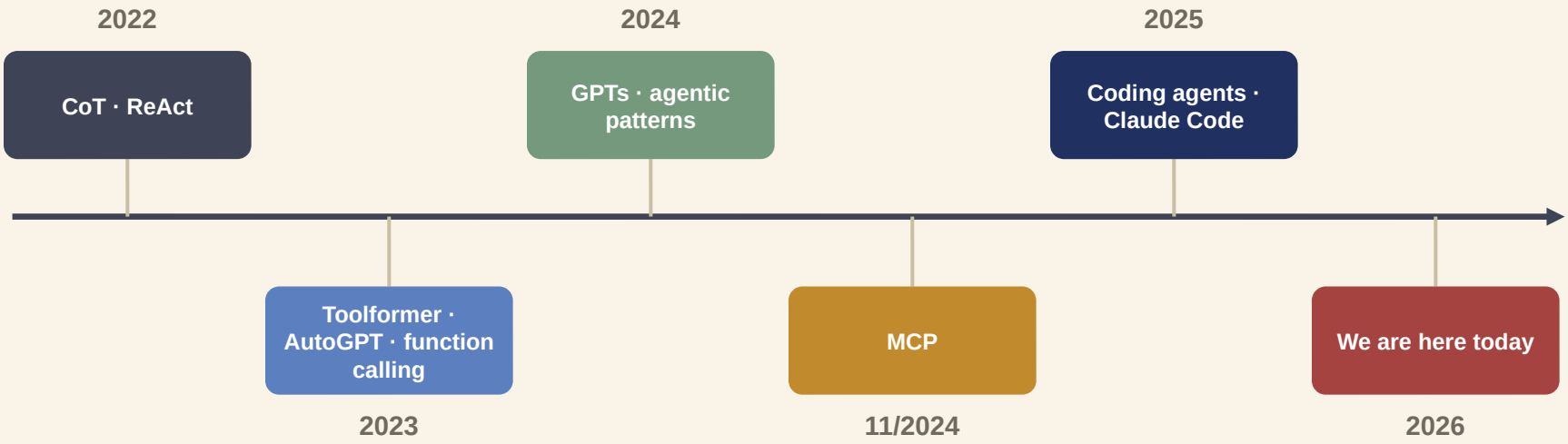


Computer use

agents that click and type

How we got here

The agent era, in six stops



The papers that started it

A research field younger than your degree



Park et al. (2023) — Generative Agents: 25 LLM agents living in a sandbox town

2021 **WebGPT (OpenAI)**
Answers by browsing — acting, not just writing

2022 **Chain-of-Thought · Wei et al.**
Reasoning happens in tokens

2022 **ReAct · Yao et al.**
Reason and act, interleaved

2023 **Toolformer · Schick et al.**
Models learn when to call tools

2023 **Voyager · Wang et al.**
Open-ended skill learning in Minecraft

What is an agent

Definitions · the loop · components · architectures

II

First, the dictionary

Merriam-Webster, as the book opens — Lanham, ch. 1.1

Acts

“One that acts or exerts power”

Causes

“Something that produces or can produce an effect”

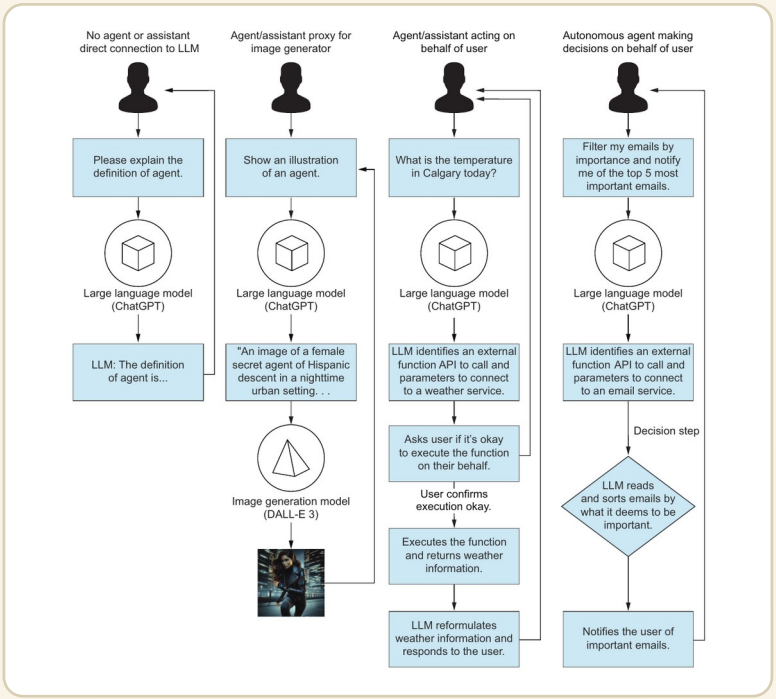
Serves

“A means by which a guiding intelligence achieves a result”

In this course, agent ≡ assistant. In Reinforcement Learning, the “agent” was the learner; here it is software that achieves results on your behalf — through an LLM.

Four ways to use an LLM

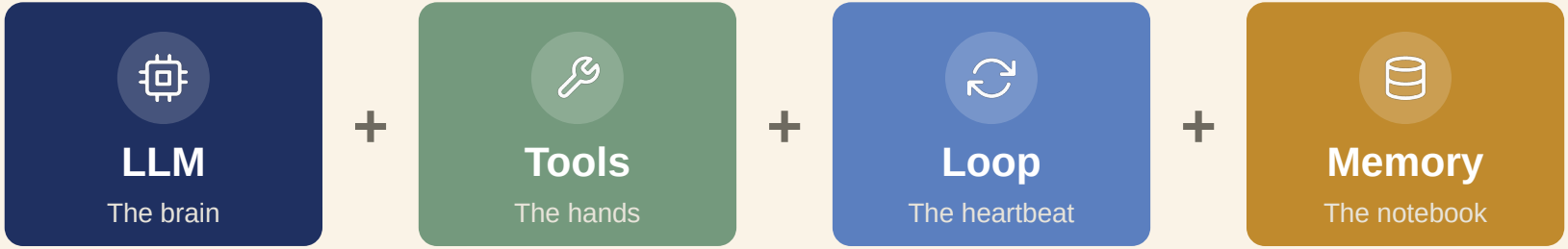
Autonomy grows to the right



Lanham, AI Agents in Action — fig. 1.1

1. **Direct** You ask, it answers
2. **Proxy** It rewrites your ask for another model
3. **Agent / assistant** Picks a function, asks approval, runs it
4. **Autonomous** Plans and decides on its own

Our working definition

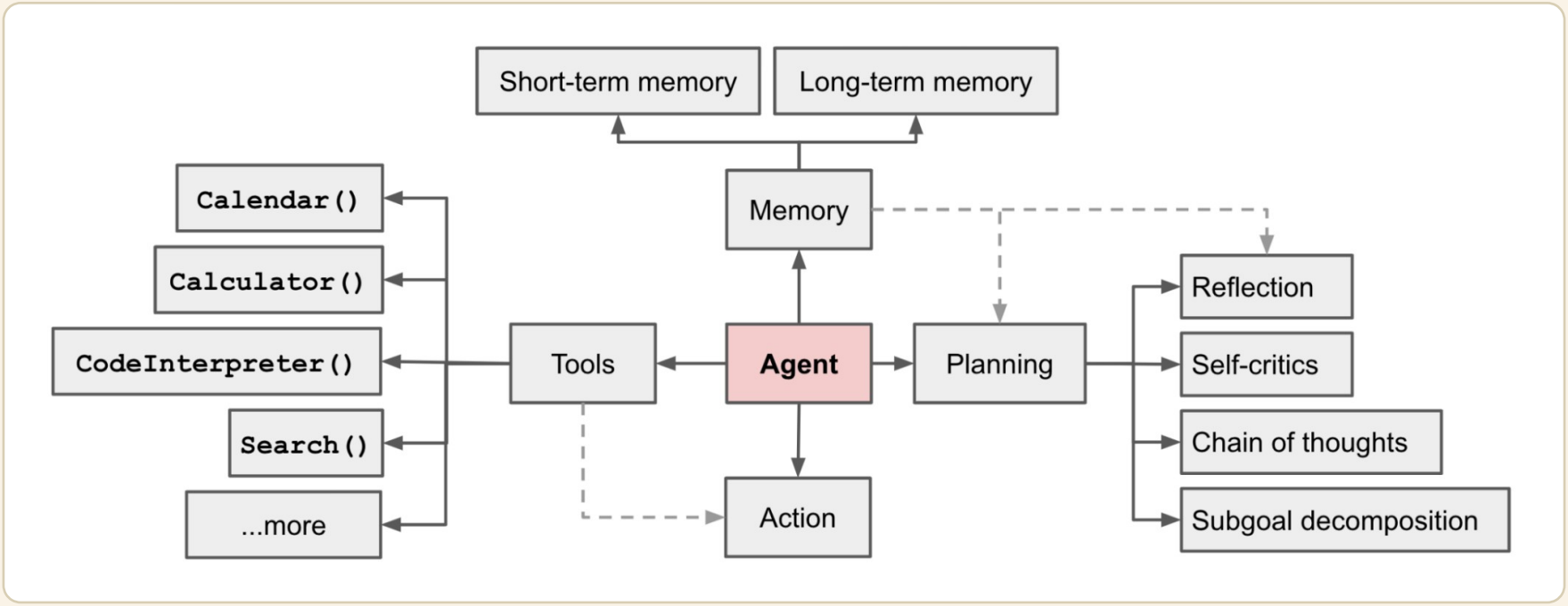


> ***"A system that uses an LLM, in a loop, to decide which actions to take toward a goal."***
>
> adapted from Lanham, AI Agents in Action (2025), ch. 1

Memory can wait — at first, ours is just the message list.

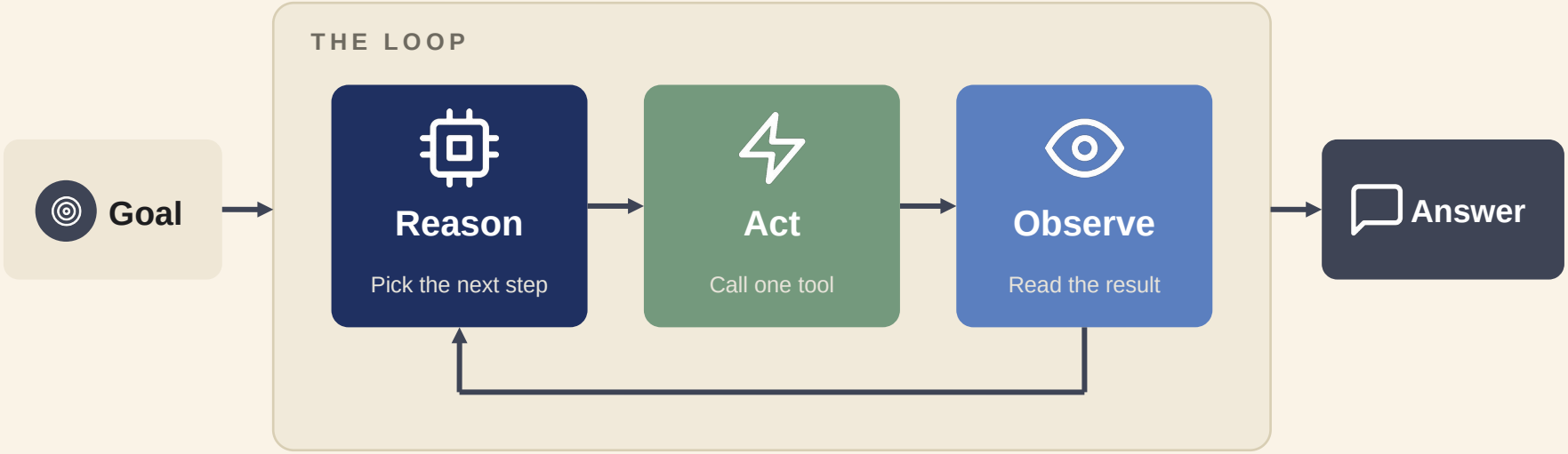
The classic map

Lilian Weng (2023) — LLM Powered Autonomous Agents



lilianweng.github.io/posts/2023-06-23-agent

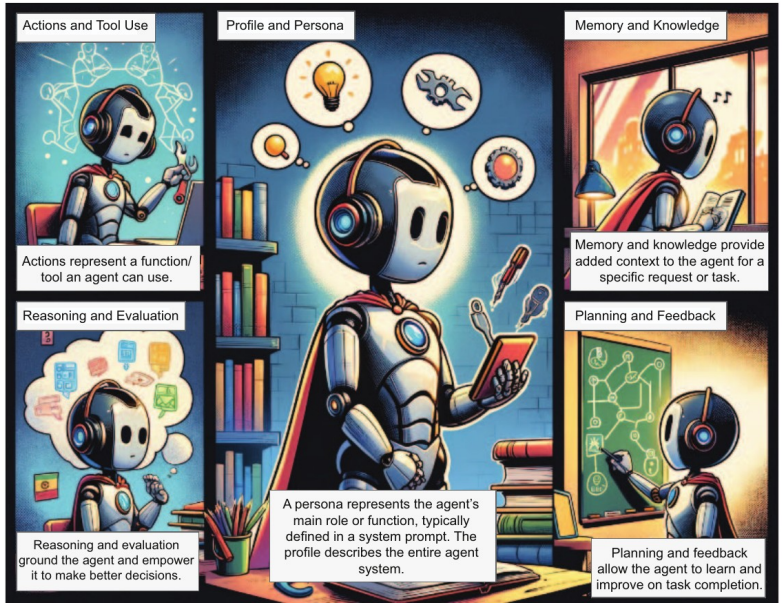
The agentic loop



Exit: when the model replies with text instead of a tool call. *Each pass appends messages to the context.*

The five components of an agent

The persona at the core; everything else extends it



Lanham, AI Agents in Action — fig. 1.3

- **Profile & persona** The system prompt — who it is
- **Actions & tools** Its action space
- **Memory & knowledge** Context on demand (RAG)
- **Reasoning & evaluation** Think, then check
- **Planning & feedback** Goals into tasks; replan

Today: persona + actions. Memory · reasoning & planning.

Inside the system prompt

```
You are PitWall, a car-buying assistant.

Tone: friendly, direct, no jargon.

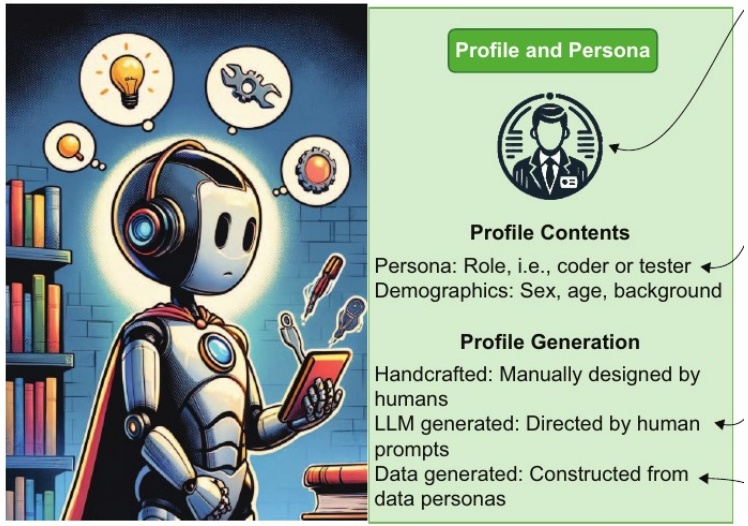
Always search_cars() before recommending;
never invent a listing.

If the budget is missing, ask for it
before searching.
```

- **Role** who it is
- **Voice** how it speaks
- **Tool policy** when to call, what never to do
- **Behavior** how to handle the unknown

You already wrote one of these: the Modelfile, week 05.

Where personas come from



Lanham, AI Agents in Action — fig. 1.4

- **Handcrafted** Written by you — your project
- **LLM-generated** A model drafts it from your prompt
- **Data-generated** Built from data, even evolved

Memory, in practice

Short-term

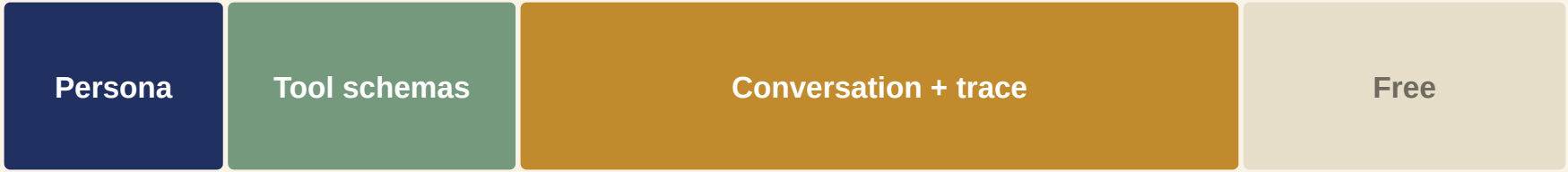
The messages list — a.k.a. the context window. Dies with the chat.

Long-term

Vector DB + retrieval — survives chats. You built this: RAG, week 06.

An agent with RAG as a tool = long-term memory on demand. (bonus idea for your project)

The context window is the RAM

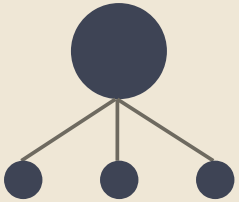


The trace grows every step — until nothing else fits

Trim, summarize, or restart. Context management returns as a Claude Code superpower — L14.

Agent architectures, at a glance

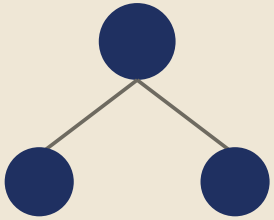
Single agent



One brain, many tools

Your project

Orchestrator



A boss delegates to specialists

Claude Code subagents

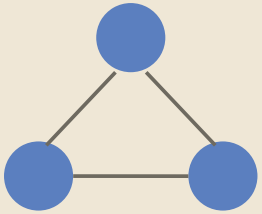
Pipeline



Each output feeds the next

Like your RAG stages

Group chat



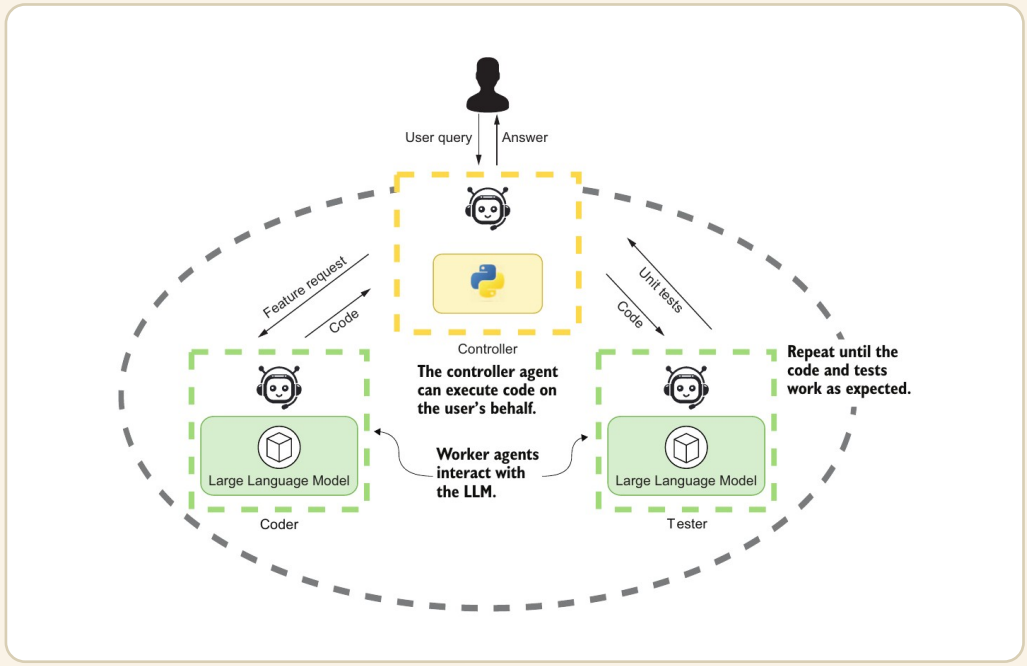
Peers argue to consensus

Eval & review patterns

Today you build the first. The rest returns in depth in lesson 12 — frameworks & multi-agent.

A dev team in miniature

One orchestrator, two workers



Lanham, AI Agents in Action — fig. 1.2

- **Controller** — Faces the user, delegates
- **Coder · Tester** — Specialist workers
- **The loop** — Repeat until tests pass

Architecture is recursive. The controller is an agent too — see Claude Code subagents.

Tools

Why · anatomy · the calling protocol · ReAct & friends

III

Why tools?

A model’s only output is text — tools give that text consequences



To know

Fresh facts, past the cutoff



To compute

LLMs guess arithmetic; tools don’t



To act

APIs, code, the real world

Toolformer (Schick et al., 2023): models can learn when to call a tool.

A tool is three things

Name

How the model refers to it

`search_cars`

Description

When to use it — the model reads this

`“Search the car dataset. Use when…”`

Parameters

The typed contract your code validates

`{max_price: number, body: enum}`

Any function can be a tool: a pandas filter, an API call, rm -rf. (choose wisely)

A tool, in full

The exact JSON your agent will carry

```
{
  "name": "search_cars",
  "description": "Search the car dataset.
     Use when the user gives a budget,
     body type or year range.",
  "parameters": {
    "type": "object",
    "properties": {
      "max_price": {"type": "number"},
      "body": {"enum": ["SUV","sedan","hatch"]},
      "min_year": {"type": "integer"}
    },
    "required": ["max_price"]
  }
}
```

- **Name** What the model writes back
- **Description** How the model chooses
- **Parameters** What your code validates

Same shape in MCP

The description is a prompt

✗ Vague

```
"search": "searches cars"
```

Called for everything — or never

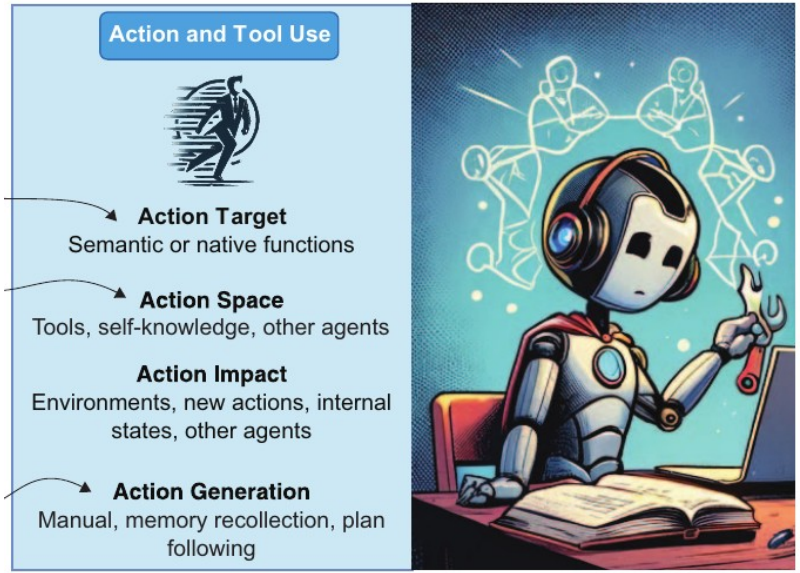
✓ Specific

```
"Search the car dataset. Use when the
user
gives budget, body type or year.
Returns ≤ 20 listings, sorted by price."
```

Right tool, right arguments, right time

The model picks tools by reading descriptions. Write them like prompts — because they are.

The design space of actions



Lanham, AI Agents in Action — fig. 1.5

- **Target** What kind of function it hits
- **Space** Tools, self-knowledge, other agents
- **Impact** What it changes in the world
- **Generation** Manual, recalled, or planned

Read tools, write tools

Read

`search · detail · compare`

No side effects — safe to retry, safe to be wrong

Your project is all read

Write

`send · buy · delete`

Side effects — confirm first, log always

Where agent safety lives

This split comes back as permissions in Claude Code — L14.

Tool calling, under the hood

1

You send

The chat so far
+ tool schemas

2

Model emits JSON

```
{"tool":
"search_cars",
 "args": {"max_price":
 80000}}
```

3

Your code runs it

The function executes — the model just waits

4

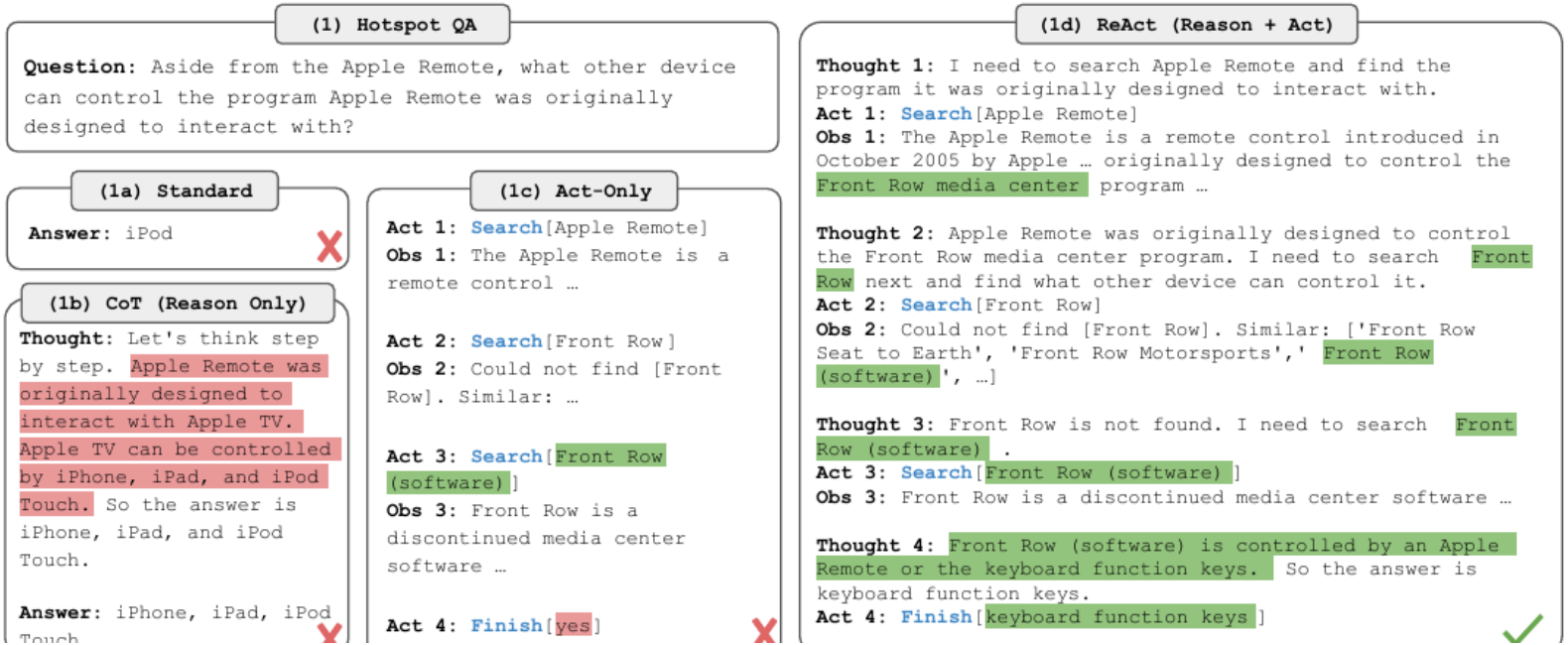
Result goes back

Appended as a message; The loop continues

The model never executes anything. It writes JSON — your loop does the work.

ReAct: reason, then act

Yao et al., ICLR 2023 — four prompting paradigms, one HotpotQA question



Yao et al. (2023), fig. 1 — arxiv.org/abs/2210.03629

ReAct, on our problem

The trace your agent will produce

```
Thought: budget R$80k, family car — space and year first
Action:  search_cars({"max_price": 80000, "body": "SUV"})
Observation: 12 listings · newest are c031, c107

Thought: the two newest look close — compare them
Action:  compare(["c031", "c107"])
Observation: c107 — newer, fewer km, same price

Answer:  “I would shortlist c107: 2022, 38k km…”
```

Thinking before acting

Verbalized thoughts lead to better tool choices

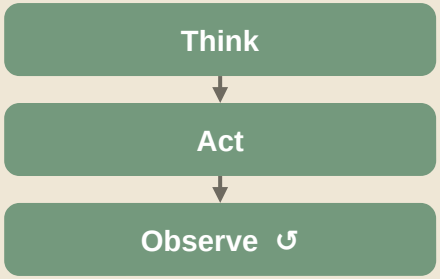
The trace is your window

Read it to debug why the agent did what it did

The pattern behind most agent loops

ReAct is one loop among several

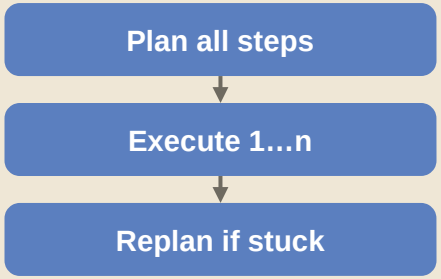
ReAct · interleaved



Adapts at every step — one LLM call per step

Yao et al., 2023

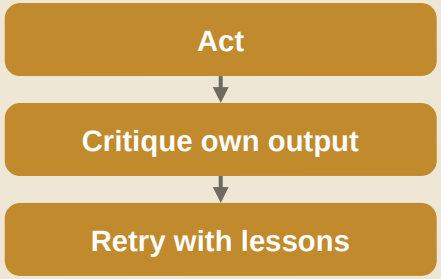
Plan-and-Execute



Cheap and predictable — brittle when the world surprises

AutoGPT · BabyAGI

Reflexion · self-critique



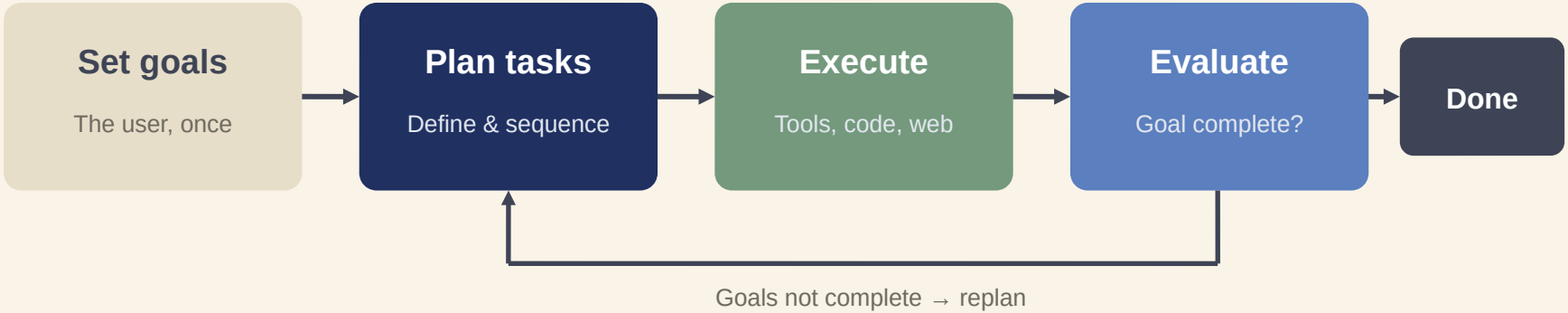
Quality climbs — paid for with extra calls

Shinn et al., 2023

All of them are prompt policies over the same while-loop. The loop stays; the policy changes.

2023: AutoGPT goes feral

Lanham, fig. 1.9 — the first viral autonomous agent



Why it went viral

Planning + iteration over GPT-4, unsupervised

Why it flopped

Drift, loops, runaway cost — no guardrails

The lesson

Autonomy is earned with evaluation

Where agents break



Infinite loops

Same action, again and again → cap the steps



Made-up inputs

Invented arguments → validate (Pydantic)



Silent errors

Tool fails quietly → errors become observations



Error propagation

One bad step poisons the rest → replan

An agent is a system — it fails like one. Design for failure.

The price of agency



Cost & latency

One task = many LLM calls



Context overflow

The Thought–Action trace fills the window fast



Prompt injection

Tool results are data, not instructions

Mitigate: cap steps · trim history · validate inputs · log every trace

Do you even need an agent?



One LLM call

USE WHEN

The task fits in a single prompt

EXAMPLE

classify radio messages
week 05



A workflow

USE WHEN

The steps are known in advance

EXAMPLE

your RAG pipeline
week 06



An agent

USE WHEN

The path depends on what it finds

EXAMPLE

“Find me a car under R$80k”
today

Start on the left. Move right only when the task demands it.

“Find the simplest system that works.” — Anthropic, Building Effective Agents (2024)

References & further reading

Micheal Lanham — AI Agents in Action (Manning, 2025)
chapter 1 (definitions, components, architectures) and chapter 5 (actions)

Yao et al. (2023) — ReAct: Synergizing Reasoning and Acting in LMs
arxiv.org/abs/2210.03629

Shinn et al. (2023) — Reflexion: Language Agents with Verbal RL
arxiv.org/abs/2303.11366

Lilian Weng (2023) — LLM Powered Autonomous Agents
lilianweng.github.io/posts/2023-06-23-agent

Andrew Ng (2024) — Agentic Design Patterns
deeplearning.ai · The Batch, four-part series

Anthropic (2024) — Building Effective Agents
anthropic.com/research/building-effective-agents · workflows vs. agents

Ollama docs — tool support
ollama.com/blog/tool-support · what the notebook uses

next · Nov 4 — Agents II: MCP, a standard for tools